SOFTWARE CHANGE IMPLEMENTATION REPORT

CR026: Forgot Password & OTP Password Reset

Field	Details
Change Request ID	CR026
Change Request Title	Forgot Password & Password Reset via Email OTP
Student Name	C. Lavasujan
Student ID	24ICT06
Implementation Status	Implemented
Report Date	14 October 2026

Change ID note: The supplied Markdown body identifies this feature as CR-002, while this report is requested as CR026. Confirm the official change-request identifier against your change register before submission.

1. Change Request Summary

1.1 Business Requirement

The Student Management System previously lacked a self-service password recovery mechanism. CR026 documents an end-to-end password reset flow that lets a user request recovery using a registered email address, receive a one-time passcode (OTP), provide a new strong password, and return to the login page after a successful reset.

1.2 Existing Behaviour

Before this change, users did not have the described self-service recovery path and needed a way to reset forgotten passwords without an administrator manually changing the account password. The supplied implementation notes describe the recovery flow and the database, backend, email, and frontend additions used to support it.

- The login form includes a “Forgot Password?” link to the recovery page.
- A registered email initiates a one-time OTP reset process; an unregistered email is rejected with “This mail is not registered.”
- OTP values are hashed before storage, expire after 10 minutes, and are subject to a maximum of three failed attempts.
- Reset requests collect the OTP, a new strong password, and matching confirmation before completing the reset.
- Environment files contain email-service settings and are configured to be excluded from version control.

1.3 Requested Change

Implement a complete email-based forgot-password and reset-password workflow, with verification, expiration, rate limiting, secure password updating, user feedback, and configuration safeguards.

- Validate the submitted email against registered accounts and reject unknown addresses.
- Generate a secure six-digit OTP, email it, and store only its SHA-256 hash with a 10-minute expiry.
- Limit failed verification to three attempts and invalidate the token when the limit is reached.
- Validate OTP length, strong-password complexity, and matching confirmation; on success, update the bcrypt hash, clear reset tokens, and redirect to login.

2. Implementation

2.1 Implementation Summary

The supplied implementation report describes a full password recovery feature spanning the frontend, backend, database, and SMTP email delivery. The login form links to ForgotPasswordPage.jsx; the recovery endpoint checks the registered email, creates a six-digit OTP, stores its SHA-256 hash with a 10-minute expiry, and dispatches an email through Nodemailer. ResetPasswordPage.jsx supports OTP entry, strong-password and confirmation checks, failed-attempt feedback, and successful redirection to /login. The backend password hash is updated with bcrypt after the reset is verified.

2.2 Affected Files

File	Change	Purpose
.gitignore	Modified	Excludes .env, .env*.local, backend/.env, and frontend/.env from Git.
database/schema.sql	Modified	Adds the email column to users and creates password_reset_tokens.
database/migration_cr002.sql	Created	Standalone schema migration script with rollback support.
backend/package.json	Modified	Adds the Nodemailer dependency for email delivery.
backend/.env	Modified	Holds SMTP/email configuration values; real credential values are intentionally not reproduced here.
backend/src/utils/emailService.js	Created	Creates SMTP transporter and HTML/plaintext OTP email templates.
backend/src/models/userModel.js	Modified	Adds reset-token create, lookup, attempt increment, and cleanup methods.
backend/src/models/studentModel.js	Modified	Links student records and user accounts through email.
backend/src/controllers/studentController.js	Modified	Persists email during student account creation.
backend/src/controllers/authController.js	Modified	Implements forgotPassword, verifyOtp, and resetPassword handlers.
backend/src/routes/authRoutes.js	Modified	Exposes /forgot-password, /verify-otp, and /reset-password endpoints.
backend/src/scripts/runMigration.js	Created	Runs and verifies the schema migration.
backend/src/scripts/testResetFlow.js	Created	Automated end-to-end script for the reset flow; execution output was not included in the supplied Markdown.
frontend/src/services/authService.js	Modified	Adds forgotPassword, verifyOtp, and resetPassword API methods.
frontend/src/components/LoginFormSection.jsx	Modified	Adds the Forgot Password? navigation link below the password field.
frontend/src/pages/ForgotPasswordPage.jsx	Modified	Implements email entry, request/error handling, resend timer, and reset navigation.
frontend/src/pages/ResetPasswordPage.jsx	Modified	Implements OTP, password strength, confirmation validation, and reset flow.

File	Change	Purpose
frontend/src/styles/login.css	Modified	Styles success alerts, OTP inputs, and password-strength indicators responsively.

2.3 Database Changes

The supplied source identifies these database modifications:

- users: add email VARCHAR(255) NULL UNIQUE. Existing user email values are backfilled from linked student records to support the new flow.
- password_reset_tokens: create a table containing id, user_id, token_hash, attempts, expires_at, used_at, and created_at; the user relationship uses a foreign key with ON UPDATE CASCADE and ON DELETE CASCADE.
- Migration: database/migration_cr002.sql provides a standalone migration with rollback support; backend/src/scripts/runMigration.js runs the migration process.

2.4 API and Email-Service Changes

Endpoint	Purpose
/api/auth/forgot-password	Validate a registered email and start an OTP reset request.
/api/auth/verify-otp	Verify the submitted email/OTP and apply the attempt-limiting behaviour described in the source.
/api/auth/reset-password	Complete a verified reset with a new strong password and confirmation.

The route paths are taken from the supplied report. Exact request/response schemas beyond the documented email error are not reproduced because the Markdown did not specify them.

3. Technical Implementation

3.1 OTP Generation, Storage, and Expiry

- The reset flow generates a cryptographically secure six-digit numeric OTP.
- Only the SHA-256 hexadecimal hash is stored in password_reset_tokens; the raw OTP is sent through the email channel rather than stored in the database.
- The token expires after 10 minutes, enforced using expires_at.
- A newly requested OTP removes any previous unused reset tokens for that user.
- A successful reset marks the token used and clears unused reset tokens.

3.2 Password Update and Attempt Limiting

- A maximum of three failed verification attempts is allowed; after the third invalid attempt, the reset token is invalidated.
- The frontend requires a six-digit OTP, a password of at least eight characters, at least one uppercase letter, one lowercase letter, one number, one special character, and matching confirmation.
- After successful OTP verification and password checks, the backend updates the password using bcrypt and redirects the user to /login after success feedback.
- An unregistered email is rejected with HTTP 404 and the message “This mail is not registered.”

3.3 Frontend Workflow

LoginFormSection.jsx provides a Forgot Password? link. ForgotPasswordPage.jsx checks email format, submits to /api/auth/forgot-password, displays server errors, and allows transition to /reset-password while carrying the target email in navigation state. ResetPasswordPage.jsx accepts the email, six-digit OTP, new password, and confirmation; it displays a password-strength indicator and reports remaining attempts when OTP submissions fail.

3.4 Configuration and Security

- SMTP is configured through environment variables EMAIL_USER, EMAIL_PASS, SMTP_HOST, SMTP_PORT, and SMTP_SECURE. The source supports TLS on port 465 or STARTTLS on port 587.
- Sensitive configuration values are not included in this report. Keep them in local environment configuration and never commit them to source control.
- The supplied Markdown states that .env files were checked as untracked using git status and are ignored by .gitignore.
- OTP hashes, attempt counts, expiry timestamps, and used_at support verification, expiry, rate limiting, and single-use enforcement.
- The temporary OTP_model/ prototype is described as superseded by the native Node.js implementation.

3.5 Implementation Logic

- The user selects Forgot Password? and enters the registered email address.
- The backend validates the email against MySQL. If no account matches, it returns 404 with “This mail is not registered.”
- For a registered account, the backend generates a six-digit OTP, stores its SHA-256 hash with a 10-minute expiry, and sends the OTP email through Nodemailer/SMTP.
- The user enters the OTP and a new strong password with matching confirmation.
- Invalid OTP submissions increment the attempt count; the token becomes invalid after three failed attempts.
- After a successful reset, the password hash is updated with bcrypt, reset tokens are invalidated/cleared, confirmation is displayed, and the user is redirected to the login page.

4. Testing & Verification

4.1 Testing Approach

The supplied Markdown identifies backend/src/scripts/testResetFlow.js as an automated end-to-end test script intended to validate the complete reset flow. It does not include the script execution output, a test pass count, or detailed browser/environment information. The test matrix below preserves the implementation behaviours specified in the source without claiming unprovided test-run results.

Testing types to cover: functional, positive, negative, token-expiry, rate-limit, password-validation, database/migration, SMTP email delivery, security configuration, and regression testing.

Environment / Browser: Not specified in the supplied Markdown.

4.2 Test Cases

Test ID	Scenario	Expected Result	Recorded Result / Evidence Gap	Status
TC-01	Request reset with a registered email	A six-digit OTP is generated, hashed, stored with a 10-minute expiry, and emailed.	Flow is described in the source; test script output not supplied.	Evidence needed
TC-02	Request reset using an unregistered email	HTTP 404 with “This mail is not registered.”	Response and message are explicitly described in the source.	Documented
TC-03	OTP token persistence and expiry	Only SHA-256 hash is stored and token expires after 10 minutes.	Schema and token behaviour are specified in the source.	Documented
TC-04	Submit incorrect OTP repeatedly	Failed attempts are counted; after three failures the token is invalidated.	Attempt limit and invalidation are specified in the source.	Documented
TC-05	Submit valid OTP and strong matching password	Password is updated with bcrypt, unused tokens are cleared, and user returns to login.	Successful reset behaviour is described; execution output not supplied.	Evidence needed
TC-06	Weak password or mismatched confirmation	Reset is blocked unless password meets complexity and confirmation matches.	Client-side rules are specified; execution evidence not supplied.	Verify / attach
TC-07	Request a replacement OTP	Previous unused reset tokens are purged when a new OTP is requested.	Token cleanup behaviour is specified in the source.	Documented
TC-08	Review environment-file protection	Sensitive .env files are ignored and remain untracked.	The source states this was verified via git status; do not screenshot or expose secrets.	Reported verified
TC-09	Run automated reset-flow script	End-to-end script validates the complete flow.	Script path is supplied; actual console output and pass count are absent.	Attach output
TC-10	Run database migration	Schema contains users.email and password_reset_tokens without dropping tables.	Migration script and runner are listed; execution log was not supplied.	Attach output

4.3 Test Summary

Field	Result
End-to-end test script	Created: backend/src/scripts/testResetFlow.js
Script execution output	Not included in the supplied Markdown
Passed / Failed count	Not specified
Pass rate	Not calculable from the supplied material
Verification status	Implementation behaviours documented; attach actual test output before reporting a test pass rate.

5. Acceptance Criteria Verification

ID	Acceptance Criterion	Verification Method	Status / Notes
AC-001	Registered email can initiate password reset.	Email lookup / forgot-password flow	Implemented per source
AC-002	Unknown email is rejected with HTTP 404 and the documented message.	Negative email-request check	Specified in source
AC-003	OTP is six digits, cryptographically generated, and dispatched by email.	Email service and reset-flow check	Implemented per source
AC-004	Only SHA-256 OTP hash is persisted and token expires in 10 minutes.	Database schema / token check	Implemented per source
AC-005	Three failed OTP attempts invalidate the token.	Invalid OTP / rate-limit check	Implemented per source
AC-006	Reset requires a strong password and matching confirmation.	Reset form validation	Rules documented; execution evidence needed
AC-007	Successful reset updates password using bcrypt and returns user to login.	End-to-end reset and login check	Implemented per source; execution output needed
AC-008	Used and previous unused reset tokens are invalidated/cleared.	Token cleanup check	Implemented per source
AC-009	Frontend and backend reset endpoints are connected.	Service/routes integration check	Files and endpoints documented
AC-010	Environment secrets are excluded from version control.	.gitignore and git status inspection	Reported verified in source
AC-011	Existing user records support email lookup through linked student records.	Migration / data-backfill check	Backward-compatibility approach documented

Verification note: "Implemented per source" means the supplied Markdown describes the functionality as implemented. It is not a substitute for attaching actual test execution results, migration logs, or screenshots when those are required for assessment.

6. Evidence

Replace each screenshot placeholder with evidence captured from the actual application or test environment. Do not include SMTP passwords, app passwords, full active OTP values, or real account passwords in the report. Use a test mailbox/account and redact sensitive values before submission.

Evidence 01 - Forgot Password link and request form

Related: AC-001, AC-009

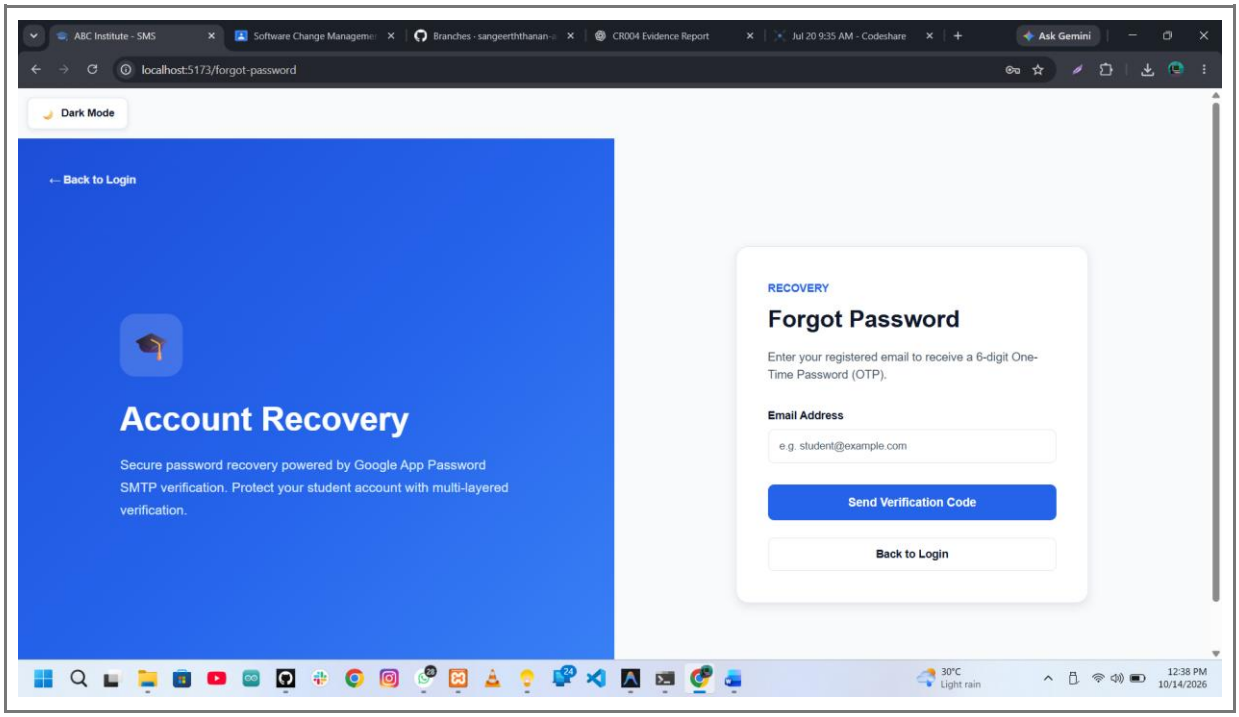


Figure 1: Login page showing the Forgot Password? link and the recovery form with a registered test email entered.

Evidence 02 - Unregistered email rejection

Related: AC-002

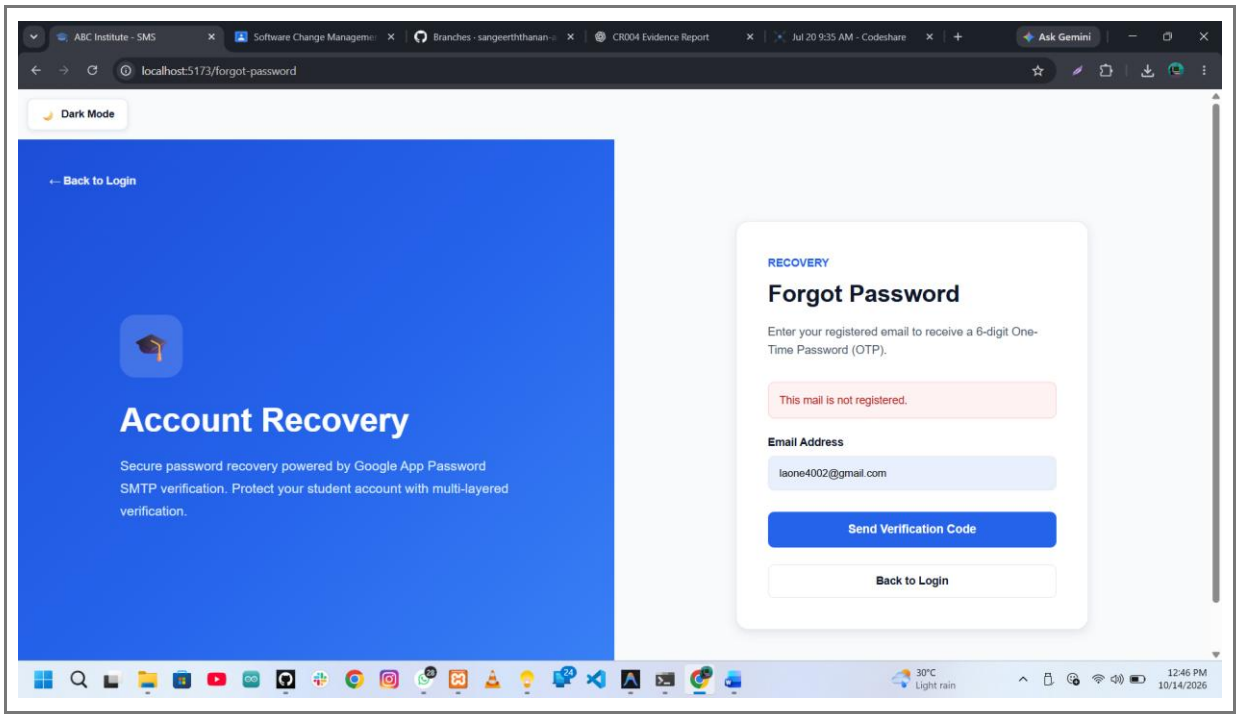


Figure 2: Forgot Password page or API client showing HTTP 404 and the message "This mail is not registered."

Evidence 03 - OTP email received

Related: AC-003, AC-004

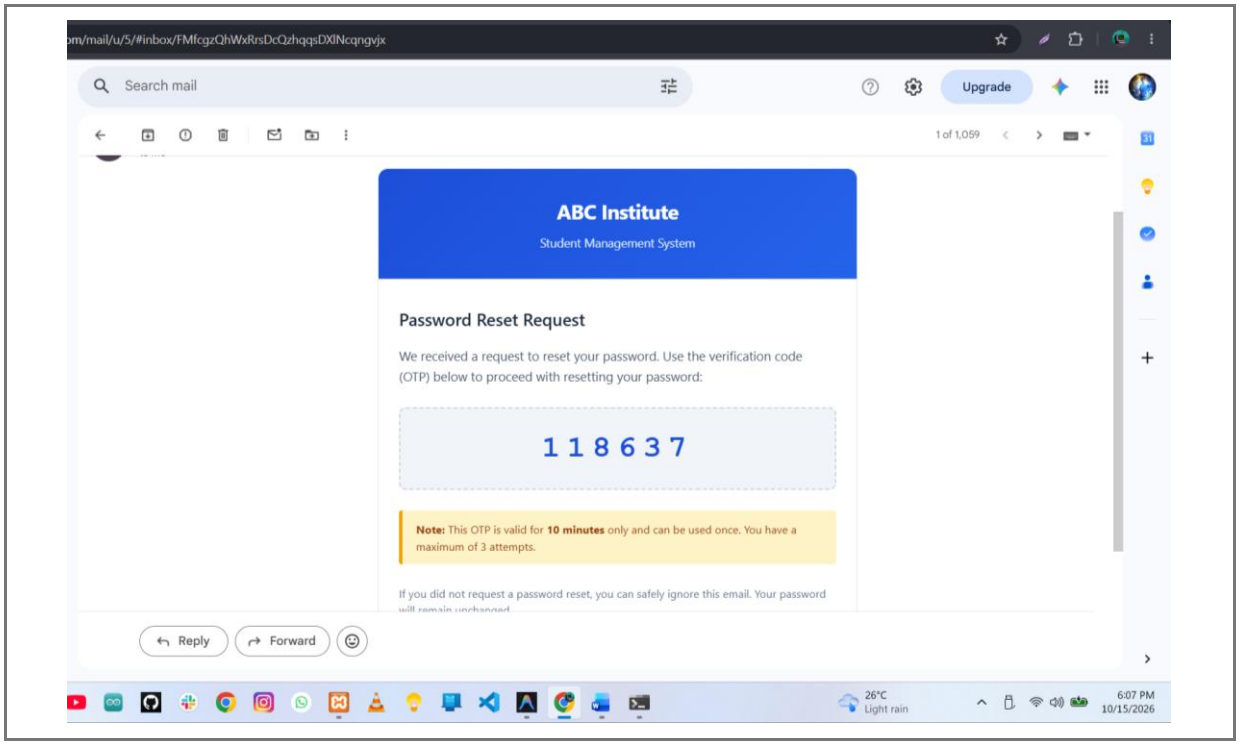


Figure 3: Test inbox showing the reset email template and 10-minute expiry/security information; redact or obscure the active OTP.

6. Evidence (continued)

Evidence 04 - Reset password form and validation

Related: AC-006, AC-009

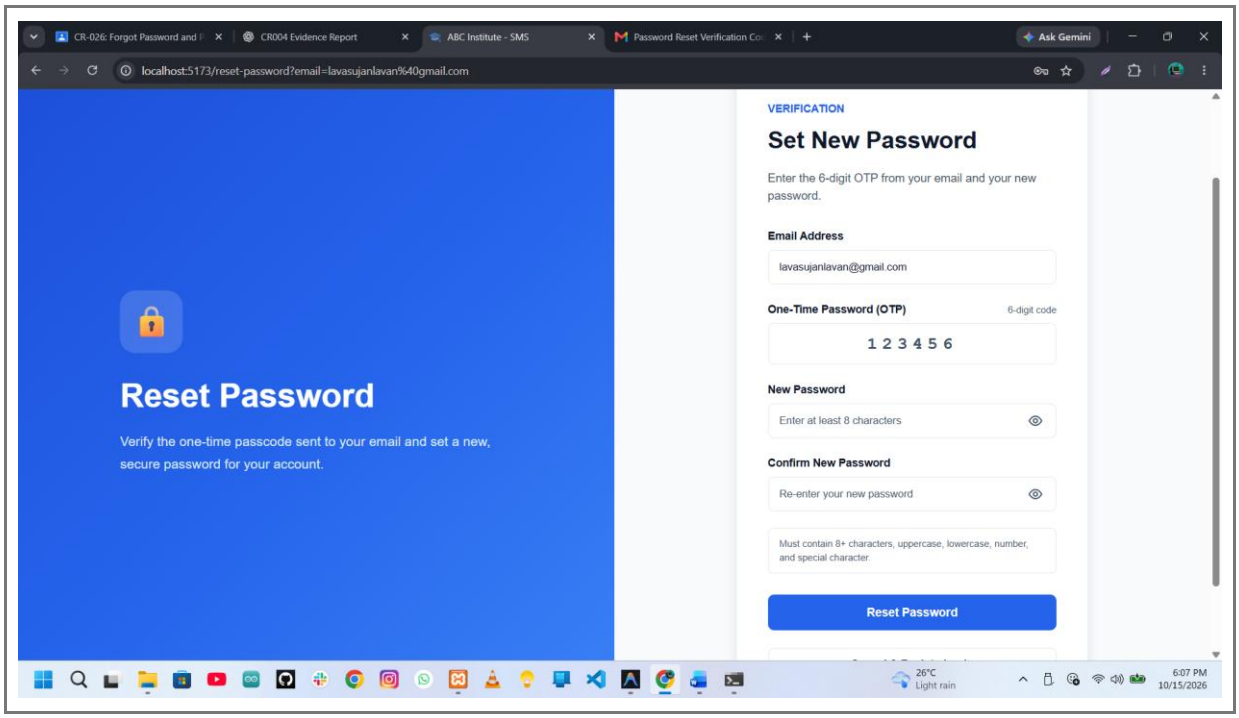


Figure 4: Reset Password page showing email, six-digit OTP input, strong-password guidance/meter, and password confirmation fields; use test values.

Evidence 05 - OTP attempt limit

Related: AC-005

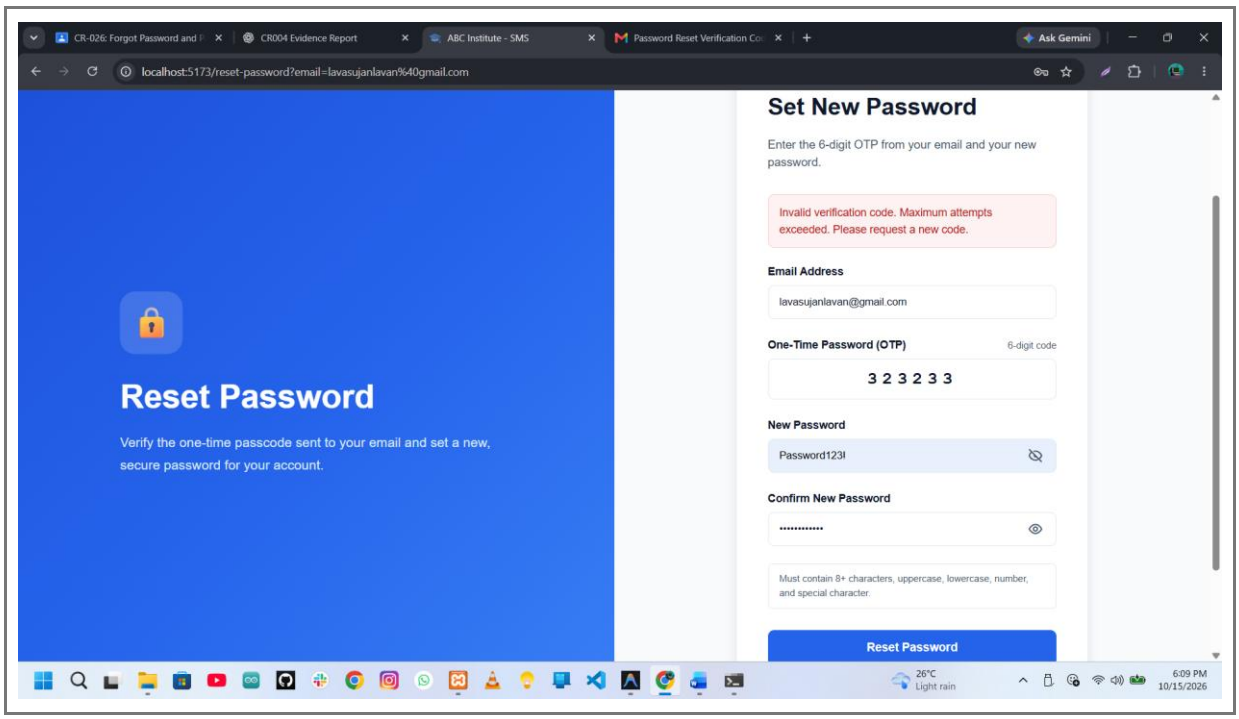


Figure 5: UI or API evidence showing failed OTP attempts and token invalidation after the third unsuccessful attempt.

Evidence 06 - Successful password reset

Related: AC-007, AC-008

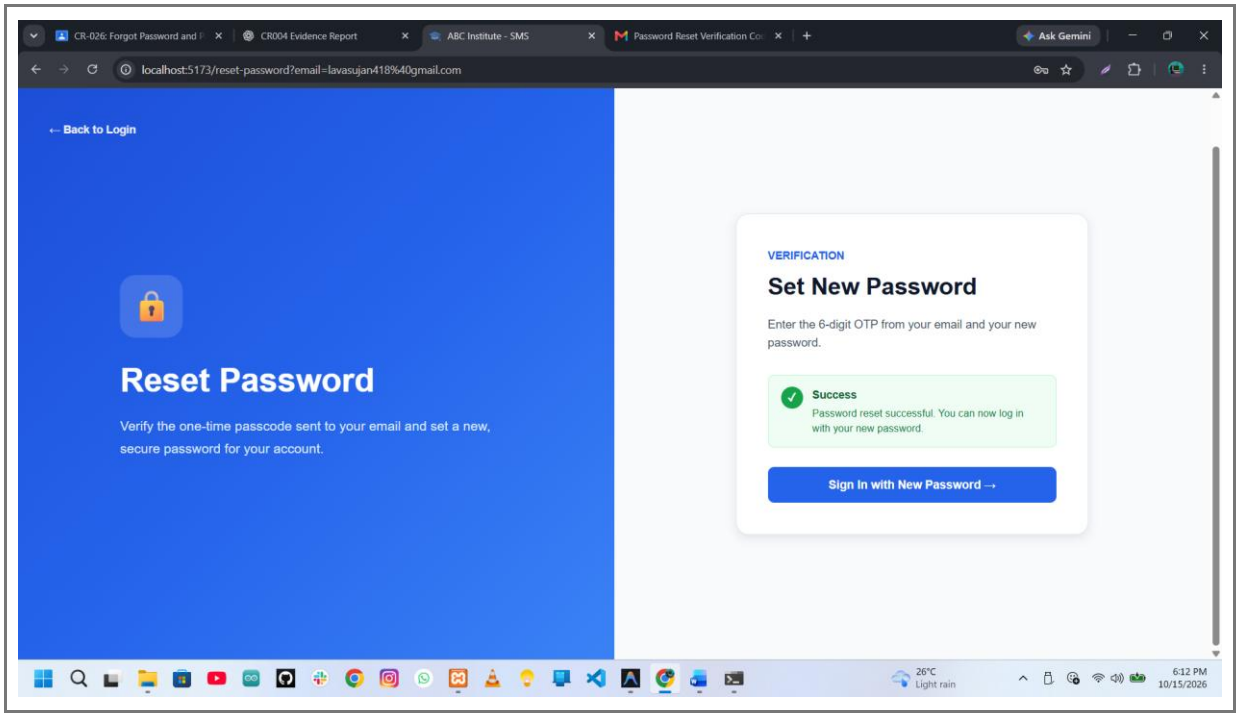


Figure 6: Success confirmation and redirect to the login page after a successful reset; then verify login with the new test password.

Appendix A - Additional Technical Evidence

Evidence 07 - Migration / database schema evidence

Related: AC-004, AC-008, AC-011

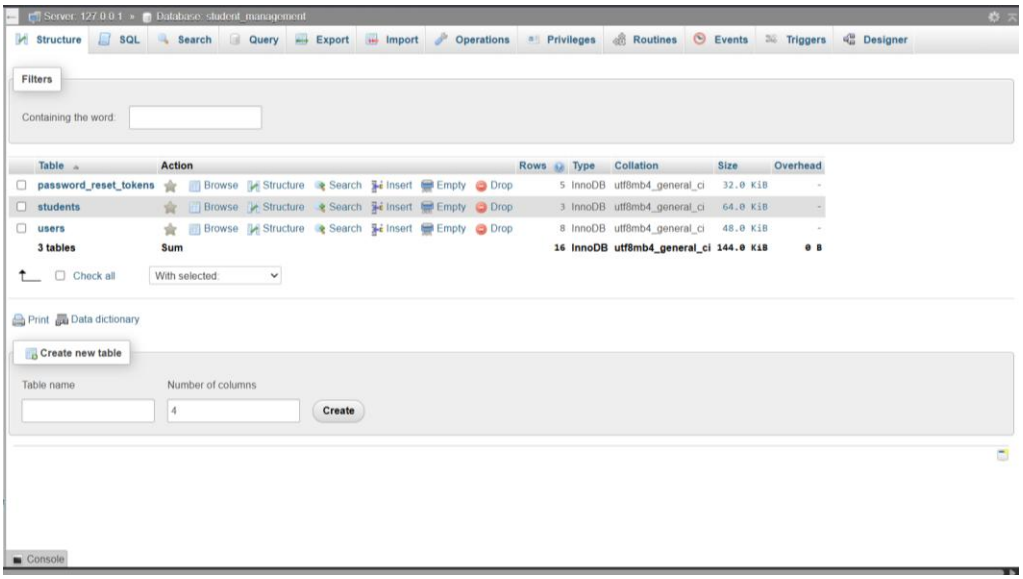


Figure 7: Migration-runner output or database schema view showing users.email and password_reset_tokens columns, with no credential values displayed.

Evidence 08 - Automated end-to-end test output

Related: AC-001 - AC-010

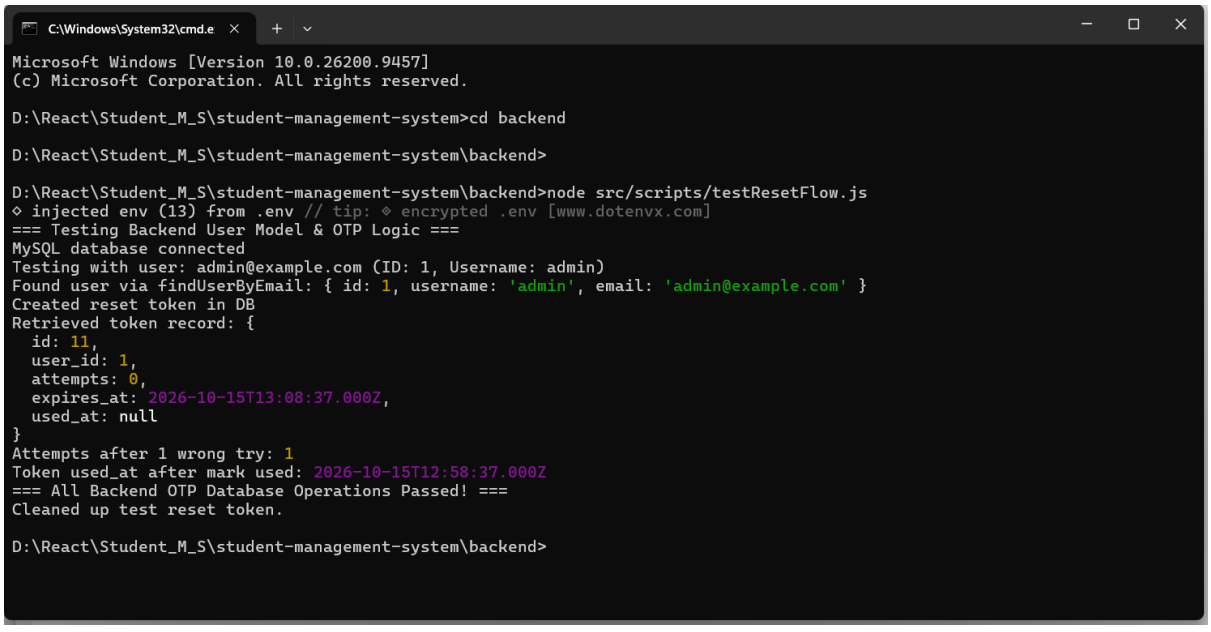


Figure 8: Terminal output from backend/src/scripts/testResetFlow.js showing the actual executed checks and pass/fail results.

7. Final Result

Item	Result
Requirements	Implementation requirements described in the supplied source are represented in this report.
Implementation	Reported as implemented in the supplied Markdown.
Testing	End-to-end test script is listed; execution output and pass count were not supplied.
Acceptance Criteria	Implementation coverage documented; attach relevant test evidence for final verification.
Security Configuration	Report omits secret values; environment files are stated to be ignored by Git.
Evidence	Screenshot and technical-evidence placeholders are ready for insertion.
Final Status	Implementation documented; final submission evidence and official CR ID should be confirmed.

7.1 Final Summary

The submitted source describes an end-to-end Forgot Password and email OTP reset feature for the Student Management System. The documented flow validates registered email addresses, generates a six-digit OTP, stores only its SHA-256 hash with a 10-minute expiry, sends the code through Nodemailer/SMTP, limits failed attempts to three, validates new-password complexity and confirmation, updates the password using bcrypt, invalidates reset tokens, and redirects the user to login. The implementation touches the schema, migration scripts, user/student models, authentication controller/routes, email service, frontend service/pages, styles, and environment-file protection.

7.2 Issues / Limitations

- The source Markdown uses CR-002 in its title, but this report is prepared as CR026 per request. Confirm the official change-request ID before submission.
- The source lists an automated end-to-end test script but does not provide execution output or a pass/fail summary. Attach the actual run output rather than claiming a pass rate without evidence.
- Screenshots and migration/test logs must be inserted into the evidence sections before final submission.
- Sensitive SMTP credentials from the source Markdown have been intentionally omitted from this document. Rotate/revoke any real app password that was included in a shared file, and keep replacement credentials only in local environment configuration.

Student Name: C. Lavasujan
Student ID: 24ICT06
Date: 14 October 2026
Signature: _____

~ END OF REPORT ~